

- Multi-Cloud & DevOps Architect — Interview Q&A (Part 2)
 - Table of Contents
 - Section 6: CI/CD Pipelines & DevSecOps {#section-6}
 - Q39. Design an enterprise CI/CD pipeline for a microservices application deployed to EKS. Walk through every stage.
 - Q40. How do you implement branch strategies and pipeline governance for 50 teams?
 - Q41. Compare Jenkins, GitHub Actions, GitLab CI, and Azure DevOps. When do you choose each?
 - Q42. How do you implement progressive delivery (blue-green, canary) across multiple clouds?
 - Q43. How do you integrate SAST, DAST, SCA, and container scanning into CI/CD without making it a bottleneck?
 - Q44. How do you manage secrets in CI/CD pipelines across multiple clouds?
 - Q45. Explain artifact management strategy across multiple clouds and teams.
 - Q46. How do you implement supply chain security for container images?
 - Q47. How do you implement environment promotion with quality gates across dev → staging → production?
 - Section 7: Observability, SRE & Incident Response {#section-7}
 - Q48. What is your approach to defining SLOs and SLIs for a multi-service platform?
 - Q49. Design a full observability stack for a microservices platform. What do you collect, store, and alert on?
 - Q50. How do you handle incident response in a multi-cloud environment?
 - Q51. How do you implement distributed tracing across microservices and clouds?
 - Q52. Explain your approach to capacity planning and performance engineering.
 - Section 8: FinOps & Cloud Cost Engineering {#section-8}
 - Q53. How do you reduce AWS compute costs by 40% without impacting reliability?
 - Q54. How do you implement chargeback/showback reporting for 50 engineering teams?
 - Q55. How do you set up budget alerts and automated cost controls?
 - Section 9: Cloud Security & Compliance {#section-9}

- Q56. How do you design a secrets management architecture for a multi-cloud environment?
- Q57. How do you implement cloud security posture management (CSPM) across AWS, Azure, and GCP?
- Q58. How do you handle IAM privilege escalation risk in AWS?
- Q59. How do you achieve SOC2 and PCI-DSS compliance in AWS?
- Q60. How do you implement encryption strategy across a multi-cloud environment?
- Section 10: Migration & Modernization {#section-10}
 - Q61. You need to migrate 200 legacy applications to AWS. How do you approach this?
 - Q62. How do you containerize a legacy Java monolith for Kubernetes?
 - Q63. How do you architect a serverless event-driven redesign from a synchronous monolith?
 - Q64. How do you handle data migration and database modernization as part of cloud migration?
- Section 11: Leadership, Architecture Governance & Soft Skills {#section-11}
 - Q65. How do you write an Architecture Decision Record (ADR)?
 - Q66. How do you run a technical architecture review for a proposal from another team?
 - Q67. How do you mentor engineers and build a high-performing cloud/DevOps team?
 - Q68. How do you handle a disagreement with a senior stakeholder about an architectural decision?
 - Q69. You've just joined an organization with significant cloud technical debt. How do you prioritize and address it?
 - Q70. How do you manage the balance between speed (developer velocity) and safety (governance/compliance)?
 - Q71. Describe your approach to writing runbooks and documentation that engineers actually use.
- Escalation
- Post-Incident

Multi-Cloud & DevOps Architect — Interview Q&A (Part 2)

Table of Contents

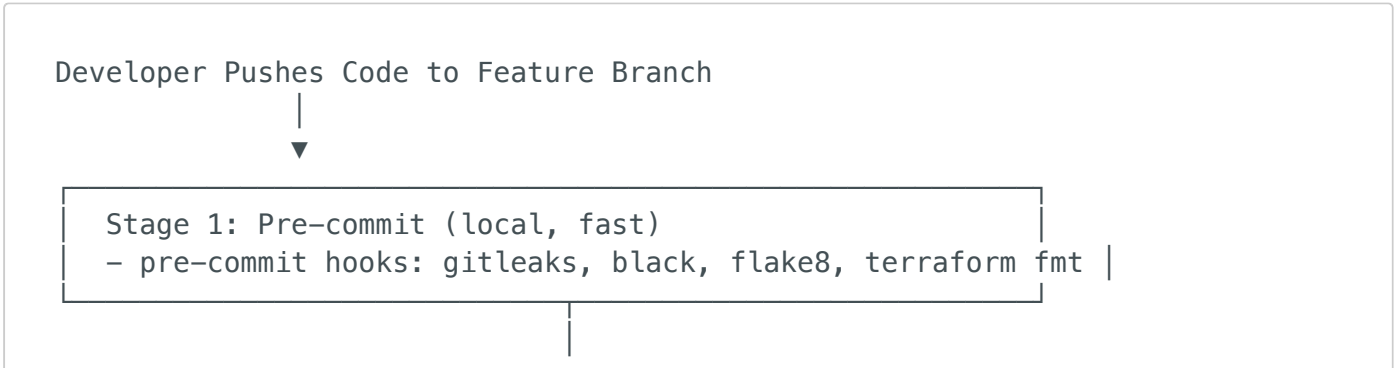
- [Section 6: CI/CD Pipelines & DevSecOps \(Q39–Q47\)](#)
- [Section 7: Observability, SRE & Incident Response \(Q48–Q55\)](#)
- [Section 8: FinOps & Cloud Cost Engineering \(Q56–Q60\)](#)
- [Section 9: Cloud Security & Compliance \(Q61–Q68\)](#)
- [Section 10: Migration & Modernization \(Q69–Q74\)](#)
- [Section 11: Leadership, Architecture Governance & Soft Skills \(Q75–Q82\)](#)

Section 6: CI/CD Pipelines & DevSecOps {#section-6}

Q39. Design an enterprise CI/CD pipeline for a microservices application deployed to EKS. Walk through every stage.

Answer:

Complete pipeline architecture:



▼ PR opened → GitHub Actions triggered

Stage 2: Code Quality & Security (< 5 min)

- Gitleaks: secret scanning entire git history
- SonarQube/SonarCloud: SAST, code coverage gate
- Hadolint: Dockerfile linting
- Checkov: IaC security scan
- OPA/Conftest: Terraform plan policy check

Stage 3: Build & Test (< 10 min)

- Unit tests + coverage report
- docker build (multi-stage, minimal base image)
- docker sbom (SBOM generation via syft)

Stage 4: Artifact Security (< 8 min)

- Trivy: scan built image (CVE check, OS + app layer)
- OWASP Dependency-Check: SCA for dependencies
- Block if CRITICAL CVEs found; WARN on HIGH
- Sign image: cosign + Sigstore (supply chain security)

Stage 5: Push & Register (immutable tags)

- Push to ECR with commit SHA tag: image:abc1234
- Attestation: SBOM attached to image in registry
- Update Helm chart values.yaml in manifest repo

Stage 6: Deploy to Staging (ArgoCD GitOps)

- ArgoCD detects manifest change → syncs staging cluster
- Smoke tests: API health check, contract tests
- Performance baseline: k6 load test

Manual approval gate (optional)

Stage 7: Progressive Deploy to Production

- Argo Rollouts: canary → 5% → 25% → 100%
- Prometheus analysis: error rate < 0.1%, p99 < 500ms
- Automatic rollback if analysis fails

GitHub Actions pipeline (abbreviated):

```
name: CI/CD Pipeline
on:
  push:
    branches: [main]
  pull_request:
    branches: [main]
```

```

permissions:
  contents: read
  id-token: write # For OIDC authentication
  security-events: write # For SARIF upload

jobs:
  security-scan:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
        with:
          fetch-depth: 0 # Full history for gitleaks

      - name: Gitleaks - Secret Scan
        uses: gitleaks/gitleaks-action@v2
        env:
          GITHUB_TOKEN: ${ secrets.GITHUB_TOKEN }

      - name: SonarCloud - SAST
        uses: SonarSource/sonarcloud-github-action@master
        env:
          GITHUB_TOKEN: ${ secrets.GITHUB_TOKEN }
          SONAR_TOKEN: ${ secrets.SONAR_TOKEN }

      - name: Run Unit Tests
        run: |
          pip install -r requirements.txt
          pytest --cov=src --cov-report=xml tests/

  build-scan-push:
    needs: security-scan
    runs-on: ubuntu-latest
    steps:
      - name: Configure AWS credentials via OIDC
        uses: aws-actions/configure-aws-credentials@v4
        with:
          role-to-assume: arn:aws:iam::${ vars.AWS_ACCOUNT_ID }:role/github-
actions-cicd
          aws-region: us-east-1

      - name: Build Docker image
        run: |
          IMAGE_TAG=${ github.sha }
          docker build -t $ECR_REGISTRY/payments-api:$IMAGE_TAG .

      - name: Trivy - Image Scan
        uses: aquasecurity/trivy-action@master
        with:
          image-ref: ${ env.ECR_REGISTRY }/payments-api:${ github.sha }
          format: sarif
          output: trivy-results.sarif
          exit-code: 1 # Fail on CRITICAL
          severity: CRITICAL,HIGH

      - name: Upload Trivy Results to GitHub Security Tab
        uses: github/codeql-action/upload-sarif@v3

```

```

with:
  sarif_file: trivy-results.sarif

- name: Sign image with cosign
  run: |
    cosign sign --yes $ECR_REGISTRY/payments-api:${{ github.sha }}

- name: Push to ECR
  run: |
    docker push $ECR_REGISTRY/payments-api:${{ github.sha }}

deploy-staging:
  needs: build-scan-push
  if: github.ref == 'refs/heads/main'
  runs-on: ubuntu-latest
  steps:
    - name: Update manifest repo
      run: |
        cd k8s-manifests
        yq e '.spec.template.spec.containers[0].image = "${{
env.ECR_REGISTRY }}/payments-api:${{ github.sha }}"' \
          -i environments/staging/payments/deployment.yaml
        git commit -am "deploy: payments-api ${{ github.sha }}"
        git push

```

Q40. How do you implement branch strategies and pipeline governance for 50 teams?

Answer:

Trunk-based development with short-lived feature branches:

```

main (trunk)
├─ feature/TICKET-123-add-payment-method (< 2 days)
├─ feature/TICKET-124-fix-auth-bug (< 2 days)
└─ hotfix/TICKET-125-critical-db-fix (< 4 hours)

```

Why trunk-based over GitFlow:

- GitFlow creates long-lived branches → merge conflicts, integration hell
- Trunk-based: small, frequent commits → easier review, faster feedback
- Feature flags replace long-lived feature branches (deploy disabled code)

Branch protection rules (GitHub):

```
# Repository: .github/CODEOWNERS
# Platform team owns infrastructure code
/terraform/          @platform-team
/k8s/                 @platform-team
/.github/workflows/   @platform-team @security-team

# Payments team owns their code
/services/payments/   @payments-team

# CODEOWNERS auto-requests review from relevant team
```

Required status checks:

```
Before merge to main:
  ✓ security-scan (must pass)
  ✓ unit-tests (must pass; coverage gate > 80%)
  ✓ terraform-plan (if .tf files changed)
  ✓ 1 approving review (from CODEOWNERS)
  ✓ No unresolved comments
  ✓ Branch is up to date with main
```

Pipeline governance at org level:

1. Reusable workflows — All teams call centrally maintained workflows:

```
# Team workflow calls central workflow
jobs:
  security-scan:
    uses: org/central-workflows/.github/workflows/security-scan.yml@main
    with:
      language: python
    secrets: inherit # Pass secrets to reusable workflow
```

2. Required workflows — Org-level policy: these workflows MUST run on all repos:

- `security-scan.yml` (Gitleaks + Trivy)
- `terraform-policy.yml` (Checkov + OPA, if .tf files exist)

3. Pipeline metrics — Track DORA metrics centrally:

- Deployment frequency, lead time, change failure rate, MTTR
- Teams with poor metrics get prioritized coaching

Q41. Compare Jenkins, GitHub Actions, GitLab CI, and Azure DevOps. When do you choose each?

Answer:

Feature	Jenkins	GitHub Actions	GitLab CI	Azure DevOps
Hosting	Self-hosted (you manage infra)	GitHub-hosted or self-hosted runners	GitLab-hosted or self-hosted	Azure-hosted or self-hosted agents
Cost	Free OSS; infra cost	Free tier; \$0.008/min per runner	Free tier; GitLab Ultimate expensive	Free tier; parallel job cost
Ecosystem	1,800+ plugins	20,000+ Marketplace actions	Built-in; some integrations	Azure-native; good .NET support
Pipeline-as-code	Declarative/Scripted (Groovy)	YAML	YAML	YAML + Classic
Scaling	Manual (Kubernetes plugin, autoscaling)	Auto (GitHub-managed)	Auto (GitLab runners)	Auto (Azure-managed)
Security	Manual hardening required	OIDC support; secret scanning	Built-in SAST/DAST/Container scan	Integration with Defender
Multi-cloud	✔ Plugin for all clouds	✔ Actions for all clouds	✔	✔ Strong AWS + Azure
Best for	Complex orchestration,	Modern GitHub-	GitLab users; DevSecOps built-in	Microsoft/Azure shops

Feature	Jenkins	GitHub Actions	GitLab CI	Azure DevOps
	legacy	native teams		

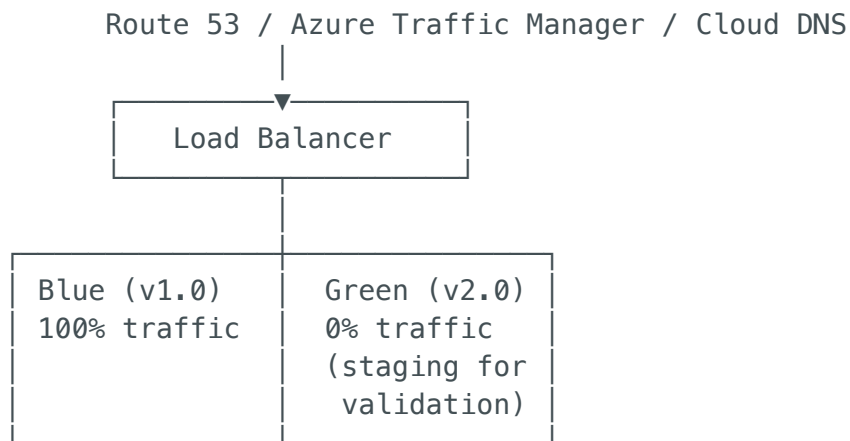
My recommendation framework:

New project, GitHub for source control	→ GitHub Actions (zero friction)
Existing Jenkins estate, complex deps	→ Keep Jenkins; modernize Jenkinsfiles to Declarative
Azure DevOps / .NET-heavy shops	→ Azure DevOps (first-class ARM/Bicep integration)
DevSecOps-first, one platform for all	→ GitLab (SAST/DAST/Container scan built-in Ultimate)
Complex multi-stage, custom orchestration	→ Jenkins or Argo Workflows

Q42. How do you implement progressive delivery (blue-green, canary) across multiple clouds?

Answer:

Blue-Green Deployment:



Deploy:

1. Deploy v2 to green (no traffic impact)
2. Run smoke tests on green directly
3. Switch ALB listener from blue → green (single API call, instant)
4. Monitor for 30 min; if issues: switch back to blue (< 5 min rollback)
5. Decommission blue after confidence

Canary with Argo Rollouts + Istio:

```

apiVersion: argoproj.io/v1alpha1
kind: Rollout
metadata:
  name: payments-api
spec:
  strategy:
    canary:
      canaryService: payments-api-canary # Istio routes here
      stableService: payments-api-stable
      trafficRouting:
        istio:
          virtualService:
            name: payments-api
            routes:
              - primary
      steps:
        - setWeight: 5 # 5% to canary
        - pause: {duration: 2m}
        - analysis: # Prometheus gate
            templates:
              - templateName: error-rate-check
        - setWeight: 25
        - pause: {duration: 5m}
        - analysis:
            templates:
              - templateName: latency-p99-check
        - setWeight: 100
  
```

```

# Analysis Template (Prometheus gate)
apiVersion: argoproj.io/v1alpha1
  
```

```
kind: AnalysisTemplate
metadata:
  name: error-rate-check
spec:
  metrics:
    - name: error-rate
      interval: 60s
      failureLimit: 2
      provider:
        prometheus:
          address: http://prometheus.monitoring:9090
          query: |
            sum(rate(http_requests_total{
              app="payments-api",
              status=~"5..",
              version="{{args.canary-hash}}"
            }[2m])) /
            sum(rate(http_requests_total{
              app="payments-api",
              version="{{args.canary-hash}}"
            }[2m])) < bool 0.01
```

Multi-cloud canary pattern:

```
AWS:   Argo Rollouts + Istio on EKS; Route53 weighted routing at global level
Azure: Argo Rollouts + Istio on AKS; Azure Front Door weighted routing
GCP:   Argo Rollouts on GKE; Cloud Load Balancing traffic splitting
Unifying layer: External DNS + Route53/Azure Traffic Manager for global routing
```

Q43. How do you integrate SAST, DAST, SCA, and container scanning into CI/CD without making it a bottleneck?

Answer:

The challenge: Security scans can add 20-40 minutes to a pipeline. Teams will disable them if they slow development.

Solution: Shift-left + Tiered scanning:

```
Tier 1 (Pre-commit, seconds):
- gitleaks: secret detection
```

- Pre-commit hooks: linting, basic security rules
- No blocking unless secrets found

Tier 2 (PR Gate, < 5 min):

- Checkov/tfsec: IaC security (fast static analysis)
 - hadolint: Dockerfile security
 - Semgrep/CodeQL: Fast SAST (parallelized)
- BLOCKS merge on: Critical findings

Tier 3 (Post-merge, < 15 min):

- Full SonarQube SAST analysis
 - OWASP Dependency-Check SCA (can be slow: 5-10 min)
 - Trivy image scan
- BLOCKS deploy on: Critical/High CVEs

Tier 4 (Scheduled, nightly):

- DAST with OWASP ZAP (against deployed staging)
 - Full license compliance scan
 - Penetration testing framework runs
- ALERTS team on: New findings

Optimization strategies:

1. **Cache dependency scan results** — OWASP Dependency-Check caches NVD database:

```
- name: Cache OWASP NVD database
  uses: actions/cache@v4
  with:
    path: ~/.m2/repository/org/owasp
    key: owasp-db-${{ hashFiles('*/pom.xml') }}
```

2. Parallel scanning:

```
jobs:
  sast:
    runs-on: ubuntu-latest
    # ... runs in parallel
  sca:
    runs-on: ubuntu-latest
    # ... runs in parallel
  image-scan:
    needs: build
    runs-on: ubuntu-latest
    # ... runs after build
# All join before deploy gate
```

3. **Risk-based exclusions** (with justification in code):

```
# .checkov.yaml
skip-check:
  - CKV_AWS_50 # Lambda public URL: justified for public API, WAF protects
  - CKV_K8S_14 # Root FS writable: legacy image, tracked in JIRA PLAT-500
```

4. **DAST asynchronously** — Run ZAP DAST against staging post-deployment; results posted to Slack, not blocking production deploy.
-

Q44. How do you manage secrets in CI/CD pipelines across multiple clouds?

Answer:

Anti-patterns I eliminate first:

- ✗ Secrets in environment variables in .github/workflows (visible in logs)
- ✗ Secrets in code (even encrypted)
- ✗ Long-lived AWS access keys in GitHub Secrets
- ✗ Passwords in docker-compose.yml
- ✗ API keys in Terraform .tfvars files committed to git

Approved patterns:

1. OIDC (Keyless) for cloud authentication:

```
# GitHub Actions → AWS (no AWS keys anywhere)
- name: Configure AWS credentials
  uses: aws-actions/configure-aws-credentials@v4
  with:
    role-to-assume: arn:aws:iam::${{ vars.ACCOUNT_ID }}:role/github-cicd
    aws-region: us-east-1
    # GitHub exchanges OIDC JWT for temporary AWS credentials
    # Credentials last 15-60 minutes, auto-expire

# GitHub Actions → Azure
- name: Login to Azure
  uses: azure/login@v2
  with:
    client-id: ${{ vars.AZURE_CLIENT_ID }}
    tenant-id: ${{ vars.AZURE_TENANT_ID }}
    subscription-id: ${{ vars.AZURE_SUBSCRIPTION_ID }}
    # Uses OIDC federated identity – no client secret
```

```
# GitHub Actions → GCP
- name: Authenticate to GCP
  uses: google-github-actions/auth@v2
  with:
    workload_identity_provider:
      projects/123/locations/global/workloadIdentityPools/...
      service_account: github-cicd@project.iam.gserviceaccount.com
```

2. HashiCorp Vault for dynamic secrets:

```
- name: Get database credentials from Vault
  uses: hashicorp/vault-action@v3
  with:
    url: https://vault.company.com
    method: jwt
    jwtGithubAudience: https://github.com/company
    role: payments-cicd
    secrets: |
      database/creds/payments-readonly username | DB_USERNAME ;
      database/creds/payments-readonly password | DB_PASSWORD
  # Vault generates a unique, short-lived DB user for this pipeline run
  # Auto-revoked when job completes
```

3. AWS Secrets Manager for application secrets:

```
# Kubernetes: External Secrets Operator syncs to K8s secrets
apiVersion: external-secrets.io/v1beta1
kind: ExternalSecret
metadata:
  name: payments-db-creds
spec:
  refreshInterval: 1h
  secretStoreRef:
    name: aws-secrets-manager
    kind: ClusterSecretStore
  target:
    name: payments-db-creds # Creates K8s secret with this name
  data:
    - secretKey: password
      remoteRef:
        key: payments/prod/db-password # AWS Secrets Manager path
        version: AWSCURRENT
```

Q45. Explain artifact management strategy across multiple clouds and teams.

Answer:

Artifact types and storage:

Artifact Type	AWS	Azure	GCP	Cross-cloud
Container images	ECR	ACR	GAR	ECR as primary, replicate
Maven/npm packages	CodeArtifact	Azure Artifacts	Artifact Registry	JFrog Artifactory
Terraform modules	S3 / Terraform Registry	Azure Storage	GCS	Private Terraform Registry
Helm charts	S3 + ECR (OCI)	ACR (OCI)	GAR (OCI)	ChartMuseum or OCI on ECR
Lambda ZIPs / function code	S3	Azure Blob	GCS	S3 primary

Multi-region image replication (ECR → ACR for disaster recovery):

```
# ECR replication to us-west-2 and eu-west-1
resource "aws_ecr_replication_configuration" "this" {
  replication_configuration {
    rule {
      destination {
        region      = "us-west-2"
        registry_id = data.aws_caller_identity.current.account_id
      }
      destination {
        region      = "eu-west-1"
        registry_id = data.aws_caller_identity.current.account_id
      }
    }
  }
}
```

Image tagging strategy:

```
Immutable tags (never overwrite):
  payments-api:abc1234def5678    (commit SHA – production deployments)
  payments-api:v2.5.1            (semantic version – release tag)

Mutable tags (latest pointer only):
  payments-api:latest            (only for local dev; never use in K8s)
```

```
manifests)
payments-api:main
```

(latest from main branch)

Image retention policy:

```
resource "aws_ecr_lifecycle_policy" "payments" {
  repository = aws_ecr_repository.payments_api.name
  policy = jsonencode({
    rules = [
      {
        rulePriority = 1
        description = "Keep last 10 production releases"
        selection = {
          tagStatus      = "tagged"
          tagPrefixList = ["v"]
          countType      = "imageCountMoreThan"
          countNumber    = 10
        }
        action = { type = "expire" }
      },
      {
        rulePriority = 2
        description = "Remove untagged images after 7 days"
        selection = {
          tagStatus      = "untagged"
          countType      = "sinceImagePushed"
          countUnit      = "days"
          countNumber    = 7
        }
        action = { type = "expire" }
      }
    ]
  })
}
```

Q46. How do you implement supply chain security for container images?

Answer:

Software supply chain threats:

- Compromised base images (SolarWinds-style attack through dependencies)
- Unsigned images (anyone can push to registry)
- Known CVEs in dependencies not caught before deployment

- Build system compromise

Defense: SLSA framework levels:

SLSA Level 1: Build process is documented (almost everyone)
SLSA Level 2: Build is version-controlled + signed (GitHub Actions default)
SLSA Level 3: Build is isolated + auditable (Tekton Chains, GitHub Actions with OIDC)
SLSA Level 4: Two-party review + hermetic builds (most enterprises target Level 3)

Implementation:

```
# 1. Generate SBOM (Software Bill of Materials)
- name: Generate SBOM
  run: |
    syft $ECR_REGISTRY/payments-api:${{ github.sha }} \
      -o spdx-json > sbom.spdx.json

    # Attach SBOM to image in registry
    cosign attach sbom \
      --sbom sbom.spdx.json \
      $ECR_REGISTRY/payments-api:${{ github.sha }}

# 2. Sign image (Sigstore/cosign - keyless signing)
- name: Sign image
  run: |
    cosign sign --yes \
      --rekor-url https://rekor.sigstore.dev \
      $ECR_REGISTRY/payments-api:${{ github.sha }}
    # Signing record stored in Sigstore's Rekor transparency log

# 3. Verify SBOM for known CVEs
- name: Scan SBOM
  run: |
    gype sbom:sbom.spdx.json --fail-on critical
```

Kubernetes admission control (only run signed images):

```
# OPA Gatekeeper: require image from approved registry
apiVersion: constraints.gatekeeper.sh/v1beta1
kind: K8sAllowedRepos
metadata:
  name: allowed-repos
spec:
  match:
    kinds:
      - apiGroups: [""]
        kinds: ["Pod"]
```

```
parameters:
  repos:
    - "123456789.dkr.ecr.us-east-1.amazonaws.com" # ECR only
    - "ghcr.io/company" # GitHub Packages
```

Connaisseur (image signature verification at admission):

```
# Reject unsigned images at Kubernetes admission
apiVersion: app.iam.policies.connaisseur.io/v1
kind: ImagePolicy
metadata:
  name: require-signed-images
spec:
  image: "*"
  keyConfig:
    name: cosign-public-key
  verify: true
```

Q47. How do you implement environment promotion with quality gates across dev → staging → production?

Answer:

Promotion pipeline:

```
Feature Branch
  | PR + CI gates (unit tests, SAST, linting)
  ▼
main branch
  | Auto-deploy to DEV (ArgoCD)
  | Integration tests run in DEV
  ▼
STAGING (promotion gate 1)
  | Performance tests (k6), contract tests (Pact), E2E (Playwright)
  | Security DAST scan
  | SLA validation: p99 latency < 500ms on staging load
  ▼
PRODUCTION (promotion gate 2)
  | Manual approval (change management for regulated environments)
  | Canary: 5% → 25% → 100% with Prometheus analysis
  ▼
PRODUCTION (full)
```

Quality gate definition (as code):

```
# .github/workflows/staging-promotion.yml
- name: Performance Test Gate
  run: |
    k6 run --vus 100 --duration 5m tests/load/api-load.js

    # Check P99 latency from k6 output
    P99=$(cat k6-results.json | jq
'.metrics.http_req_duration.values["p(99)"]')
    if (( $(echo "$P99 > 500" | bc -l) )); then
      echo "❌ P99 latency $P99ms exceeds 500ms gate"
      exit 1
    fi
    echo "✅ P99 latency $P99ms passes gate"

- name: Contract Test Gate
  run: |
    # Pact: verify all consumer contracts are satisfied
    pact verify --provider payments-api \
      --pact-broker-url https://pact.company.com \
      --provider-version ${github.sha} \
      --publish-verification-results
```

Section 7: Observability, SRE & Incident Response {#section-7}

Q48. What is your approach to defining SLOs and SLIs for a multi-service platform?

Answer:

SLI/SLO/SLA hierarchy:

```
SLI (Service Level Indicator) = the measurement
→ "What percentage of requests succeed in < 200ms?"

SLO (Service Level Objective) = your reliability target
→ "99.9% of requests succeed in < 200ms over a 30-day rolling window"

SLA (Service Level Agreement) = the contractual commitment with penalty
→ "We guarantee 99.5% availability; credit for breaches"
```

Error Budget = 100% – SLO

→ For 99.9% SLO: 0.1% = 43.8 minutes/month allowed downtime

SLI types I define for every service:

SLI Type	Formula	Example
Availability	good_requests / total_requests	HTTP 5xx rate
Latency	% requests under threshold	p99 < 500ms
Throughput	Requests per second	> 1000 RPS under load
Error rate	Failed operations / total	< 0.1% payment failures
Freshness	Age of newest data	Data < 5 min old

Prometheus-based SLO with Sloth or Pyrra:

```
# Sloth SLO definition
apiVersion: sloth.slok.dev/v1
kind: PrometheusServiceLevel
metadata:
  name: payments-api-availability
  namespace: monitoring
spec:
  service: "payments-api"
  slo:
    - name: "requests-availability"
      objective: 99.9 # 99.9% SLO
      description: "99.9% of payment requests succeed"

  sli:
    events:
      errorQuery: |
        sum(rate(http_requests_total{
          service="payments-api",
          status_code=~"5.."
        }[{{.window}}]))
      totalQuery: |
        sum(rate(http_requests_total{
          service="payments-api"
        }[{{.window}}]))

  alerting:
    name: PaymentsAPIAvailabilityBurn
    pageAlert: # Page on-call
    labels:
      severity: critical
    ticketAlert: # Create ticket
```

```
labels:  
  severity: warning
```

Error budget burn rate alerts (multi-window):

```
Fast burn (1h window): if consuming 5% error budget in 1 hour → PAGE  
Slow burn (6h window): if consuming 10% error budget in 6 hours → TICKET  
Budget exhausted alert: if < 10% remaining for month → Escalate to  
leadership
```

Error budget policy:

```
Error budget > 50% remaining:  
  → Full feature velocity; experiments allowed  
  
Error budget 10–50% remaining:  
  → Prioritize reliability work alongside features  
  
Error budget < 10% remaining:  
  → Freeze risky deployments; focus only on reliability  
  
Error budget exhausted:  
  → Stop all feature work; all hands on reliability  
  → Post-mortem required before resuming
```

Q49. Design a full observability stack for a microservices platform. What do you collect, store, and alert on?

Answer:

The three pillars + profiling:

```
METRICS (quantitative, aggregate)  
  → What is happening? Error rate, latency, throughput  
  → Tools: Prometheus + Grafana  
  
LOGS (events, discrete)  
  → What happened in detail? Error messages, audit events  
  → Tools: Loki + Grafana (or ELK for structured search)  
  
TRACES (request journey)
```

- How did a request flow through my system?
- Tools: Tempo (or Jaeger) + Grafana

PROFILES (resource usage per code path)

- Which code is consuming CPU/memory?
- Tools: Pyroscope + Grafana

Full stack architecture:

Applications

- ├ Metrics: Prometheus scrape (or push via OTEL)
- ├ Logs: stdout → Promtail/FluentBit → Loki
- └ Traces: OTEL SDK → OTEL Collector → Tempo

OTel Collector (DaemonSet):

Receivers: otlp, prometheus, k8s_events, aws_cloudwatch
Processors: batch, memory_limiter, resource_detection, k8s_attributes
Exporters: prometheusremotewrite → Mimir
 loki → Loki
 otlp → Tempo

Storage:

Metrics: Grafana Mimir (long-term Prometheus, scalable)
Logs: Loki (S3 backend, cheap long-term storage)
Traces: Tempo (S3/GCS backend)

Visualization & Alerting:

Grafana: Dashboards, Explore, Alerting
PagerDuty: On-call routing

What I instrument by default (every service, day 1):

```
# Python service: OpenTelemetry auto-instrumentation
from opentelemetry import trace, metrics
from opentelemetry.instrumentation.flask import FlaskInstrumentor
from opentelemetry.instrumentation.requests import RequestsInstrumentor
from opentelemetry.instrumentation.sqlalchemy import SQLAlchemyInstrumentor

# Auto-instruments:
# - All HTTP requests/responses (status, duration, method, path)
# - All SQL queries (query text, duration, errors)
# - All outbound HTTP calls
# Plus custom business metrics:

meter = metrics.get_meter("payments")
payment_counter = meter.create_counter(
    "payments.processed",
    unit="1",
    description="Number of payment transactions"
)
```

```
@app.route("/pay", methods=["POST"])
def process_payment():
    with tracer.start_as_current_span("process_payment") as span:
        span.set_attribute("payment.amount", amount)
        span.set_attribute("payment.currency", currency)
        span.set_attribute("payment.method", method)

        result = payment_service.charge(...)

        payment_counter.add(1, {"method": method, "status": result.status})
    return result
```

Key dashboards I build for every service:

- 1. **RED Dashboard** — Rate, Errors, Duration per service
- 2. **USE Dashboard** — Utilization, Saturation, Errors per node/resource
- 3. **Kubernetes Overview** — Pod count, restart count, node pressure
- 4. **Business Metrics** — Revenue, orders, active users (joins infra + business)

Q50. How do you handle incident response in a multi-cloud environment?

Answer:

Incident severity classification:

Severity	Definition	Response Time	Escalation
P0	Complete service outage; > \$100K/hour revenue impact	Immediate	CTO + On-call within 5 min
P1	Major feature broken; > 20% users affected	15 minutes	Engineering Lead
P2	Partial degradation; workaround exists	1 hour	Team lead
P3	Minor issue; low user impact	Next business day	Team

On-call rotation setup (PagerDuty):

Primary on-call: responds within 5 minutes
Secondary on-call: escalation after 10 minutes silence
Manager escalation: after 20 minutes
C-suite bridge: P0 only, after 30 minutes

Tools:

- PagerDuty: alert routing
- Slack: incident channel #inc-YYYY-MM-DD-[service]
- Zoom: war room for P0/P1
- Statuspage.io: external customer communication
- Jira: post-incident tracking

Incident response runbook template:

Incident Response Steps

1. Declare Incident (2 min)

- Create #inc-channel in Slack
- Assign Incident Commander (IC) – not the same person debugging
- Page secondary if needed

2. Triage (10 min)

- What is broken? (service, endpoint, region)
- Who is affected? (check dashboards, error rates, user reports)
- Is there a recent deployment? (ArgoCD history, GitHub Actions)
- Is there unusual traffic? (bot attack, spike)

3. Mitigation (goal: restore service, not fix root cause)

- Can we rollback the last deployment? (ArgoCD rollback: 2 min)
- Can we disable the feature flag? (LaunchDarkly: instant)
- Can we scale up? (Karpenter: 2-3 min)
- Can we failover to backup region? (Route53 weighted: 5 min)

4. Communication

- Every 15 min: status update to Slack channel
- Customer-facing: Statuspage update within 10 min of P0/P1 detection
- Stakeholders: email summary for P0

5. Resolution

- Service restored → mark resolved in PagerDuty
- Monitoring 30 min post-restoration before all-clear

6. Post-Mortem (within 48h for P0/P1)

- Blameless: focus on system, process, not people
- 5 Whys analysis
- Action items with owners and due dates

Multi-cloud incident tooling:

AWS: CloudWatch Alarms → SNS → PagerDuty

Azure: Azure Monitor Alerts → Action Groups → PagerDuty

GCP: Cloud Monitoring Alerts → Pub/Sub → PagerDuty

Cross-cloud correlated view: Grafana (federated query across all clouds)

Q51. How do you implement distributed tracing across microservices and clouds?

Answer:

Distributed tracing flow:

```
Client Request
|
▼ HTTP Header: traceparent: 00-{trace-id}-{parent-span-id}-01
Frontend Service (EKS)
| Creates root span
├── HTTP call to Payments API (AKS) – propagates trace context
│   ├── SQL query to RDS – child span
│   └── SQS message publish – child span
└── HTTP call to Identity Service (EKS)
    └── Redis lookup – child span
```

All spans flow → OTel Collector → Tempo → Grafana Tempo UI

Grafana shows: full trace timeline, service topology, bottleneck identification

OTel Collector configuration for multi-cloud:

```
# otel-collector-config.yaml
receivers:
  otlp:
    protocols:
      grpc:
        endpoint: 0.0.0.0:4317
      http:
        endpoint: 0.0.0.0:4318

processors:
  batch:
    timeout: 1s
    send_batch_size: 1024
```

```

memory_limiter:
  limit_mib: 512
resourcedetection:
  detectors: [eks, ecs, ec2, azure, gcp] # Auto-detect cloud metadata
k8sattributes:
  extract:
    metadata:
      - k8s.namespace.name
      - k8s.pod.name
      - k8s.deployment.name
      - k8s.node.name

exporters:
  otel:
    endpoint: tempo.monitoring.svc.cluster.local:4317
    tls:
      insecure: false
  prometheusremotewrite:
    endpoint: http://mimir.monitoring.svc.cluster.local/api/v1/push

service:
  pipelines:
    traces:
      receivers: [otel]
      processors: [memory_limiter, resourcedetection, k8sattributes, batch]
      exporters: [otel]
    metrics:
      receivers: [otel]
      processors: [memory_limiter, batch]
      exporters: [prometheusremotewrite]

```

Grafana Tempo + Loki correlation:

```

# Grafana: Jump from trace → related logs
# In Tempo, configure Loki datasource link:
{namespace="${__span.tags.k8s.namespace.name}",
  pod="${__span.tags.k8s.pod.name}"}
| json
| trace_id="${__span.traceId}"

```

Q52. Explain your approach to capacity planning and performance engineering.

Answer:

Capacity planning model:

Current state metrics (90-day p99 baseline):

- CPU utilization per service: 45% avg, 78% p99
- Memory utilization: 60% avg, 85% p99
- Requests per second: 5,000 avg, 12,000 peak (Black Friday)
- Database connections: 180 / 200 pool max at peak

Growth projections:

- Business: 30% YoY user growth
- Traffic: Linear with user growth
- Data: 50% YoY storage growth (compressible)

Capacity required:

- 12 months: $5,000 * 1.3 = 6,500$ avg RPS
- Black Friday: $6,500 * 2.4$ (historical ratio) = 15,600 peak RPS

Current headroom: $12,000 / 15,600 = 77\%$ → need to scale before next BF

Load testing strategy:

```
# k6 load test script
import http from 'k6/http';
import { check, sleep } from 'k6';
import { Rate } from 'k6/metrics';

const errorRate = new Rate('errors');

export const options = {
  stages: [
    { duration: '5m', target: 100 }, // Ramp up
    { duration: '10m', target: 1000 }, // Baseline load
    { duration: '5m', target: 5000 }, // Spike test
    { duration: '10m', target: 1000 }, // Back to baseline
    { duration: '5m', target: 0 }, // Ramp down
  ],
  thresholds: {
    http_req_duration: ['p(99)<500'], // P99 < 500ms
    errors: ['rate<0.01'], // < 1% error rate
    http_req_failed: ['rate<0.01'],
  },
};

export default function () {
  const res = http.post('https://staging.payments.company.com/v1/pay',
    JSON.stringify({ amount: 100, currency: 'USD' }),
    { headers: { 'Content-Type': 'application/json' } }
  );

  errorRate.add(!check(res, { 'status is 200': (r) => r.status === 200 }));
  sleep(1);
}
```

Section 8: FinOps & Cloud Cost Engineering {#section-8}

Q53. How do you reduce AWS compute costs by 40% without impacting reliability?

Answer:

Cost reduction levers (with typical savings):

Lever	Typical Savings	Implementation
Spot instances for non-critical workloads	60-70% vs On-Demand	EKS Spot node groups, Karpenter
Reserved Instances / Savings Plans	30-40% vs On-Demand	1-year Compute Savings Plans
Right-sizing oversized instances	20-30%	AWS Compute Optimizer recommendations
Auto-scaling with Karpenter bin packing	15-25%	Karpenter consolidation enabled
Dev environment auto-shutdown	60-70% of dev costs	Lambda + EventBridge scheduler
S3 Intelligent-Tiering for infrequent data	40-68% storage	Auto-transitions infrequent objects

Concrete example — 40% reduction:

Starting state: 100 c5.4xlarge on-demand nodes (\$0.68/hr each)
Monthly cost: $100 \times \$0.68 \times 720\text{hr} = \$48,960/\text{month}$

Step 1: Move 70% to Spot (Karpenter diversification)
30 on-demand + 70 spot (\$0.20/hr avg with diversification)
New cost: $(30 \times \$0.68 + 70 \times \$0.20) \times 720 = \$24,624/\text{month}$
Savings: 50% – Spot interruptions handled by Karpenter (re-provision in < 60s)

Step 2: Compute Savings Plans on the 30 on-demand nodes (1-year, no upfront)

Discount: 36%

Savings Plans cost: $30 \times \$0.435 \times 720 = \$9,396$

Total: $\$9,396 + \$14,400 = \$23,796/\text{month}$

Step 3: Karpenter consolidation (remove underutilized nodes)

Bin packing reduces node count from 100 → 75 equivalent units

New total: ~\$17,847/month

Final: \$17,847 vs \$48,960 original = 63.6% reduction

Key: Maintained reliability through Spot diversification + on-demand baseline

Savings Plans vs Reserved Instances:

Compute Savings Plans (recommended):

- Applies to any EC2 instance type, size, region, OS
- Also applies to Lambda and Fargate
- Flexible: change instance family without losing discount
- 1-year no-upfront ≈ 30%; 3-year all-upfront ≈ 50%

EC2 Reserved Instances:

- Locked to specific instance type + region + OS
- Convertible RIs: can exchange type; Zonal: capacity reservation
- Use for stable, predictable workloads where you won't change instance type
- RDS, ElastiCache, OpenSearch: still use RI (no Savings Plans available)

Q54. How do you implement chargeback/showback reporting for 50 engineering teams?

Answer:

Chargeback vs Showback:

Showback: "Here is what your team spent – informational, no budget impact"

Chargeback: "Here is what your team spent – deducted from your budget"

Start with showback to build visibility culture.

Move to chargeback when teams have budget autonomy.

Architecture:

```
AWS Cost & Usage Report (CUR)
Azure Cost Export
GCP Billing Export
    |
    ▼
S3 Data Lake (raw CUR data in Parquet)
    |
    ▼
dbt / Databricks (transform + normalize)
  - Join with CMDB (team → service → account mapping)
  - Apply shared cost allocation (support services, networking)
  - Calculate unit economics (cost per deployment, cost per user)
    |
    ▼
QuickSight / Grafana / Looker
  - Per-team dashboards (each team sees only their costs)
  - Monthly trend
  - Anomaly detection
    |
    ▼
Monthly cost report → Email + Slack #cloud-costs-{team}
```

Tagging enforcement:

```
# AWS Config Rule: detect untagged resources
resource "aws_config_rule" "required_tags" {
  name = "required-tags"
  source {
    owner          = "AWS"
    source_identifier = "REQUIRED_TAGS"
  }
  input_parameters = jsonencode({
    tag1Key = "Team"
    tag2Key = "Environment"
    tag3Key = "CostCenter"
  })
}

# SCP: deny resource creation without required tags
{
  "Effect": "Deny",
  "Action": ["ec2:RunInstances", "rds:CreateDBInstance",
"eks:CreateCluster"],
  "Resource": "*",
  "Condition": {
    "Null": {
      "aws:RequestTag/Team": "true"
    }
  }
}
```

Shared cost allocation:

```
# Transit Gateway costs shared across all teams by traffic volume
# Central networking account charges teams proportionally

tgw_total_cost = 5000 # USD/month
team_traffic_share = {
    "payments": 0.35, # 35% of transit traffic
    "identity": 0.15,
    "analytics": 0.50
}

# Chargeback: payments gets $1,750 of TGW cost
for team, share in team_traffic_share.items():
    allocated_cost = tgw_total_cost * share
    report[team]["networking"] += allocated_cost
```

Q55. How do you set up budget alerts and automated cost controls?

Answer:

AWS Budget configuration:

```
resource "aws_budgets_budget" "payments_team" {
  name           = "payments-prod-monthly"
  budget_type    = "COST"
  limit_amount   = "15000"
  limit_unit     = "USD"
  time_unit      = "MONTHLY"

  cost_filter {
    name = "TagKeyValue"
    values = ["Team$payments"] # Filter by Team tag
  }

  notification {
    comparison_operator = "GREATER_THAN"
    threshold           = 80
    threshold_type      = "PERCENTAGE"
    notification_type    = "ACTUAL"
    subscriber_sns_topic_arns = [aws_sns_topic.budget_alerts.arn]
  }

  notification {
    comparison_operator = "GREATER_THAN"
    threshold           = 100
  }
}
```



```
        for i in r['Instances']]:

    if instance_ids:
        ec2.stop_instances(InstanceIds=instance_ids)
        print(f"Stopped {len(instance_ids)} dev instances: {instance_ids}")
```

Section 9: Cloud Security & Compliance

{#section-9}

Q56. How do you design a secrets management architecture for a multi-cloud environment?

Answer:

Secrets management principles:

1. No secrets in code, ever (use pre-commit hooks to detect)
2. No long-lived static credentials (use dynamic secrets or short-lived tokens)
3. Least privilege (each service only accesses its own secrets)
4. Audit trail (who accessed what secret, when)
5. Automatic rotation (not manual rotation on a calendar)

HashiCorp Vault architecture for multi-cloud:

HashiCorp Vault Cluster
(HA: 3 nodes + Consul/Raft)

Auth Methods:

- AWS IAM auth
- Azure Managed Identity
- GCP service account
- Kubernetes JWT

Secret Engines:

- AWS: dynamic IAM creds
- Azure: dynamic SP creds

- Database: dynamic DB pw
- PKI: short-lived TLS certs
- KV v2: static secrets

Dynamic database credentials (eliminate shared passwords):

```
# Vault policy: each service gets unique, short-lived DB credentials
resource "vault_database_secret_backend_role" "payments_api" {
  name      = "payments-api"
  backend   = vault_database_secrets_mount.postgres.path

  db_name = "payments-db"

  creation_statements = [
    "CREATE USER \"{{name}}\" WITH PASSWORD '{{password}}' VALID UNTIL '{{expiration}}';",
    "GRANT SELECT, INSERT, UPDATE ON payments.* TO \"{{name}}\";"
  ]

  revocation_statements = ["DROP USER IF EXISTS \"{{name}}\";"]

  default_ttl = "1h"      # Credentials expire after 1 hour
  max_ttl     = "24h"
}
```

Kubernetes integration (External Secrets Operator):

```
# ESO syncs Vault → Kubernetes Secret
apiVersion: external-secrets.io/v1beta1
kind: SecretStore
metadata:
  name: vault-backend
spec:
  provider:
    vault:
      server: "https://vault.company.com"
      path: "secret"
      auth:
        kubernetes:
          mountPath: "kubernetes"
          role: "payments-api"
          serviceAccountRef:
            name: payments-api

---
apiVersion: external-secrets.io/v1beta1
kind: ExternalSecret
metadata:
  name: payments-db-creds
spec:
```

```
secretStoreRef:
  name: vault-backend
target:
  name: db-credentials
data:
  - secretKey: password
    remoteRef:
      key: secret/payments/prod/db
      property: password
```

Q57. How do you implement cloud security posture management (CSPM) across AWS, Azure, and GCP?

Answer:

CSPM tools comparison:

Tool	Coverage	Key Strength
AWS Security Hub	AWS-native	Aggregates findings from GuardDuty, Inspector, Macie, Config
Microsoft Defender for Cloud	Azure + multicloud connectors	Azure-native; integrates with Sentinel SIEM
Google Security Command Center	GCP-native	Excellent for GCP; Asset discovery
Wiz / Prisma Cloud	All three clouds	Best multi-cloud CSPM; agentless; graph-based
Prowler (OSS)	All three clouds	Free; CLI + CI/CD integration; CIS benchmarks

My CSPM architecture:

```
Wiz (or Prisma Cloud) – Unified CSPM
├── AWS: read-only IAM role with security audit permissions
│   └── Scans EC2, S3, RDS, IAM, SecurityGroups, KMS
├── Azure: App Registration with Reader role
│   └── Scans VMs, Storage, Key Vault, RBAC
```

- └─ GCP: Service Account with Security Reviewer role
 - Scans Compute, GCS, BigQuery, IAM

Findings → JIRA tickets (critical: P1, high: P2)
→ Slack alert for critical misconfigurations
→ Weekly executive report (coverage score, trend)

CIS Benchmark baseline (AWS example — key controls):

CIS AWS Foundations Benchmark v1.4:

IAM:

- ✓ 1.1: Avoid use of root account
- ✓ 1.4: Ensure no access keys for root
- ✓ 1.5: Enable MFA for root account
- ✓ 1.14: Hardware MFA for root
- ✓ 1.16: Ensure IAM policies attached to groups, not users

Logging:

- ✓ 2.1: CloudTrail enabled in all regions
- ✓ 2.2: CloudTrail log file validation enabled
- ✓ 2.3: CloudTrail logs encrypted with KMS CMK
- ✓ 2.7: CloudTrail logs delivered to CloudWatch

Networking:

- ✓ 4.1: No unrestricted SSH (port 22)
- ✓ 4.2: No unrestricted RDP (port 3389)
- ✓ 4.3: No unrestricted access to ports 0–65535

Q58. How do you handle IAM privilege escalation risk in AWS?

Answer:

Privilege escalation paths in AWS:

An attacker with limited permissions could:

1. iam:CreatePolicyVersion → Create new policy version with Admin permissions
2. iam:AttachUserPolicy → Attach Administrator policy to themselves
3. iam:CreateLoginProfile → Create console password for a powerful user
4. iam:PassRole + ec2:RunInstances → Launch EC2 with a powerful role attached
5. lambda:CreateFunction + lambda:InvokeFunction → Run code as a powerful Lambda role

Defenses I implement:

1. Deny privilege escalation actions for non-admin roles:

```
{
  "Effect": "Deny",
  "Action": [
    "iam:CreatePolicyVersion",
    "iam:DeletePolicyVersion",
    "iam:SetDefaultPolicyVersion",
    "iam:AttachUserPolicy",
    "iam:AttachGroupPolicy",
    "iam:AttachRolePolicy",
    "iam:PutUserPolicy",
    "iam:PutGroupPolicy",
    "iam:PutRolePolicy",
    "iam:CreatePolicy"
  ],
  "Resource": "*",
  "Condition": {
    "StringNotLike": {
      "aws:PrincipalArn": [
        "arn:aws:iam::*:role/platform-admin-role",
        "arn:aws:iam::*:role/terraform-execution-role"
      ]
    }
  }
}
```

2. IAM Access Analyzer — continuous policy analysis:

```
resource "aws_accessanalyzer_analyzer" "org_analyzer" {
  analyzer_name = "OrganizationAnalyzer"
  type          = "ORGANIZATION" # Analyzes cross-account access in the org
}
```

3. Permission Boundaries — cage for IAM delegation:

```
# Teams can create IAM roles, but the roles cannot exceed this boundary
resource "aws_iam_policy" "developer_boundary" {
  name = "DeveloperPermissionBoundary"
  policy = jsonencode({
    Statement = [
      {
        Effect    = "Allow"
        Action    = ["s3:*", "sqs:*", "dynamodb:*", "lambda:*"]
        Resource  = "*"
      },
    ],
  })
}
```

```

    Effect    = "Deny" # Cannot do IAM, even if directly assigned
    Action    = ["iam:*", "organizations:*", "sts:AssumeRole"]
    Resource   = "*"
  }
}
})
}

```

4. CloudTrail + GuardDuty detection:

GuardDuty finding types to alert on:

- IAMUser/AnomalousBehavior: Unusual API calls
- PrivilegeEscalation/IAMUser: IAM policy self-attachment
- UnauthorizedAccess/IAMUser.ConsoleLoginSuccess: Unusual console login

Q59. How do you achieve SOC2 and PCI-DSS compliance in AWS?

Answer:

SOC 2 Trust Service Criteria mapped to AWS controls:

SOC 2 Criteria	AWS Implementation
CC6.1 Logical access	IAM least privilege + SCPs + MFA
CC6.2 Authentication	AWS IAM Identity Center + MFA required
CC6.7 Data in transit	TLS 1.2+ enforced; no HTTP
CC6.8 Data at rest	KMS CMK encryption for S3, RDS, EBS
CC7.1 Monitoring	CloudTrail + CloudWatch + Security Hub
CC7.2 Threat detection	GuardDuty + Macie + Inspector
CC8.1 Change management	Terraform IaC + approval workflows
A1.1 Availability	Multi-AZ + auto-scaling + health checks

PCI-DSS specific requirements (card data environments):

Requirement 1: Network segmentation

- Cardholder Data Environment (CDE) in isolated VPC
- WAF on all entry points
- No direct internet access to CDE instances (all traffic via ALB/NAT)

Requirement 3: Protect stored cardholder data

- Never store PANs in plaintext; tokenization via Stripe/Braintree
- If stored: AES-256 encryption with HSM-backed keys (AWS CloudHSM)

Requirement 7: Restrict access by business need to know

- IAM: attribute-based access control
- Database: column-level security on card tables

Requirement 10: Track and monitor all access

- CloudTrail enabled in all regions
- VPC Flow Logs enabled
- RDS audit logging to CloudWatch
- Log retention: minimum 12 months, 3 months immediately available

Requirement 11: Test security systems

- Quarterly ASV scans
- Annual penetration testing
- Continuous vulnerability management (Inspector)

Automated compliance evidence collection:

```
# Use AWS Config + Security Hub for evidence
aws securityhub get-compliance-summary-by-config-rule \
  --config-rule-names pci-dss-v321-*

# Export compliance dashboard to PDF monthly for auditors
# Store evidence in S3 with Object Lock (WORM) – can't be deleted
```

Q60. How do you implement encryption strategy across a multi-cloud environment?

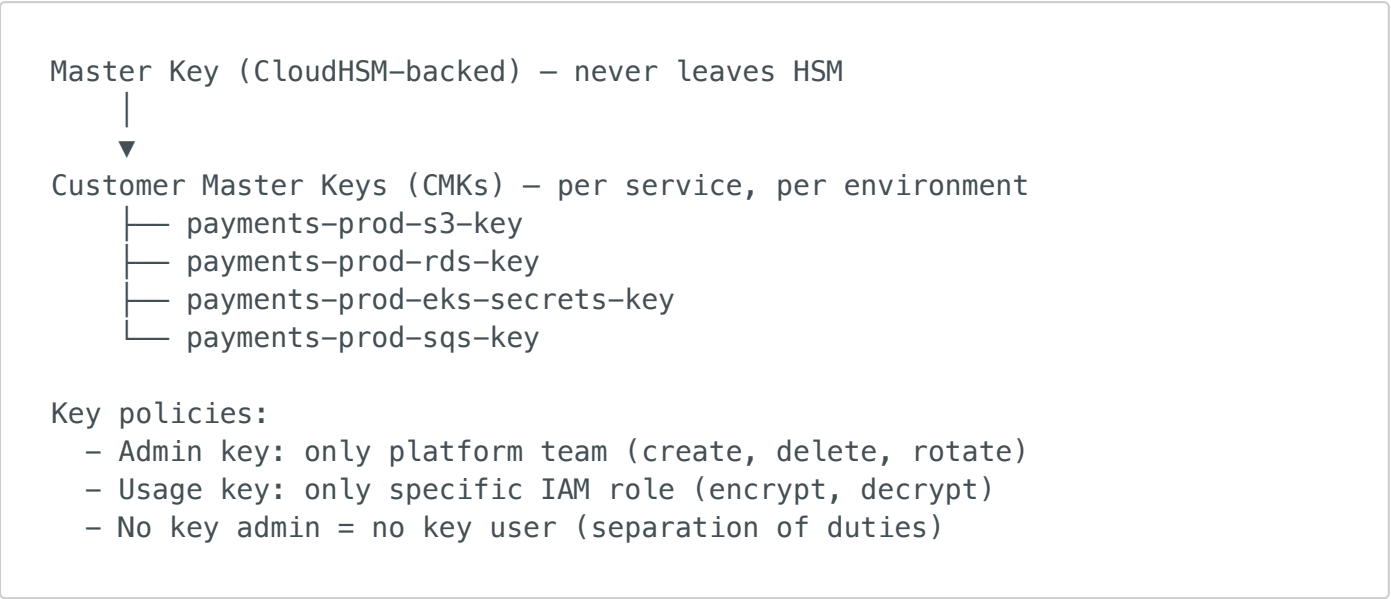
Answer:

Encryption decision matrix:

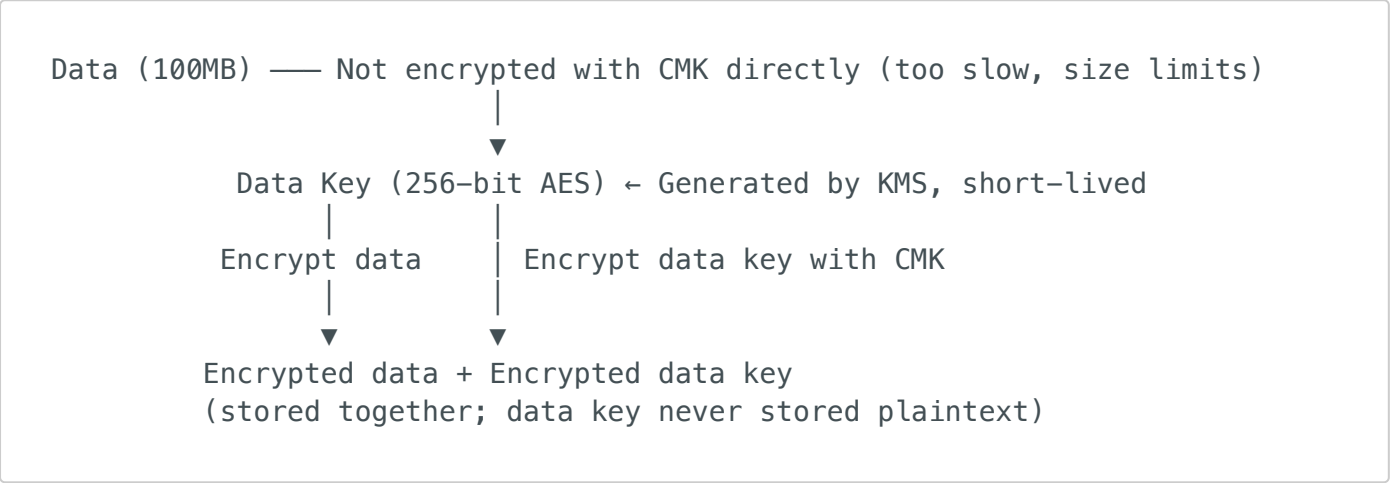
Data State	AWS	Azure	GCP
At rest (default)	SSE-S3 (AWS-managed)	Storage Service Encryption	Google-managed keys

Data State	AWS	Azure	GCP
At rest (compliance)	SSE-KMS (customer-managed CMK)	Customer-Managed Key (Azure Key Vault)	Cloud KMS CMEK
At rest (highest security)	SSE-C (customer-provided key) + CloudHSM	Azure Dedicated HSM	Cloud HSM
In transit	TLS 1.2+ (AWS managed)	TLS 1.2+	TLS 1.2+
In use	AWS Nitro Enclaves	Azure Confidential Computing	Google Confidential VMs

KMS key hierarchy design:



Envelope encryption explained:



Section 10: Migration & Modernization

{#section-10}

Q61. You need to migrate 200 legacy applications to AWS. How do you approach this?

Answer:

Migration framework — 7Rs:

Migration Strategy	Description	When to Use
Retire	Decommission	15-20%: unused, duplicate, EOL systems
Retain	Keep on-prem	10-15%: compliance, latency, mainframe-specific
Rehost (lift & shift)	Move to EC2/IaaS as-is	30-40%: fast migration, no code changes
Replatform	Minor optimization (RDS, ECS)	20-30%: gain cloud benefits, minimal code change
Repurchase	Move to SaaS	5-10%: CRM → Salesforce, HR → Workday
Refactor	Re-architect for cloud-native	10-15%: highest ROI but most effort; microservices
Relocate	Move VMs to VMware on AWS	Legacy VMware shops needing fast migration

Phased migration approach:

Phase 1: Discovery & Assessment (Months 1–3)
→ AWS Migration Hub + AWS Application Discovery Service

- Map application dependencies (what calls what)
- Score each app: Complexity (1–5) × Business Value (1–5)
- Quick wins first: low complexity, high value

Phase 2: Foundation (Months 3–6)

- Deploy Landing Zone (Control Tower)
- Set up Direct Connect
- Establish connectivity, DNS, monitoring baselines
- Migrate first wave: static apps, dev environments

Phase 3: Main Migration Waves (Months 6–18)

- Wave 1: Rehost tier-3 applications (low risk, fast)
- Wave 2: Replatform tier-2 applications
- Wave 3: Refactor tier-1 strategic applications

Phase 4: Optimize & Modernize (Months 18+)

- FinOps: rightsize, savings plans
- Decommission on-premises hardware
- Containerize rehosted apps (second pass)

Tooling:

Tool	Purpose
AWS MGN (Application Migration Service)	Block-level replication; lift-and-shift
AWS DMS	Database migration (homogeneous + heterogeneous)
AWS Schema Conversion Tool	Convert Oracle/SQL Server → Aurora PostgreSQL
AWS DataSync	Mass data transfer to S3/EFS
Terraform (my IaC)	Reproduce infrastructure as code post-migration

Q62. How do you containerize a legacy Java monolith for Kubernetes?

Answer:

Phased containerization approach:

Phase 1: Containerize as-is (Replatform)

```

# Dockerfile: Lift monolith into container (first step)
FROM amazoncorretto:17-alpine3.19

# Create non-root user (security)
RUN addgroup -S appgroup && adduser -S appuser -G appgroup

WORKDIR /app

# Copy built artifact
COPY --chown=appuser:appgroup target/payments-monolith-1.0.jar app.jar

# Run as non-root
USER appuser

# Expose metrics port for Prometheus
EXPOSE 8080 8081

# Graceful shutdown: SIGTERM handler in Spring Boot 2.3+
ENTRYPOINT ["java", \
  "-XX:+UseContainerSupport", \
  "-XX:MaxRAMPercentage=75.0", \
  "-XX:+ExitOnOutOfMemoryError", \
  "-Djava.security.egd=file:/dev/./urandom", \
  "-jar", "app.jar"]

```

Phase 2: Extract bounded contexts (Refactor to microservices)

Identify strangler-fig boundaries:

```

Payments Monolith
├── Auth module          → Extract to identity-service (separate
repo/deployment)
├── Notification module → Extract to notification-service (event-driven)
├── Reporting module    → Extract to reporting-service (read-only replica)
└── Core payments      → Remains as slimmed monolith initially

```

Strangler Fig Pattern:

```

Route /auth/* → new identity-service (via Istio VirtualService)
Route /notify/* → new notification-service
Route /* → monolith (everything else, until extracted)

```

Database migration pattern (avoid shared database):

Legacy: Single monolithic database (Oracle)

Step 1: Introduce Change Data Capture (Debezium → Kafka)
 Replicate monolith data changes to Kafka in real-time

Step 2: New services subscribe to Kafka topics
 identity-service builds its own PostgreSQL from identity-related CDC events

Step 3: Cut over

identity-service reads from its own DB; monolith continues writing

After validation: monolith stops writing identity data; service owns it

This avoids big-bang DB migration risk.

Q63. How do you architect a serverless event-driven redesign from a synchronous monolith?

Answer:

Pattern: Choreography-based event-driven architecture

Before (synchronous, tightly coupled):

```
User → API → PaymentService.process()
                        └─ EmailService.sendConfirmation() (sync call)
                        └─ InventoryService.decrementStock() (sync call)
                        └─ LoyaltyService.addPoints() (sync call)
```

Total latency: 200ms + 150ms + 100ms + 80ms = 530ms

Single failure cascades to entire flow

After (event-driven, loosely coupled):

```
User → API → PaymentLambda → EventBridge → {
  → EmailLambda (async, user doesn't wait)
  → InventoryLambda (async)
  → LoyaltyLambda (async)
}
```

API response latency: 80ms (just payment processing)

Other services decouple and retry independently

AWS serverless event-driven implementation:

```
# PaymentLambda: process payment + publish event
import boto3, json
from aws_lambda_powertools import Logger, Tracer, Metrics
from aws_lambda_powertools.metrics import MetricUnit

logger = Logger()
tracer = Tracer()
metrics = Metrics(namespace="Payments")

@logger.inject_lambda_context
@tracer.capture_lambda_handler
```

```

@metrics.log_metrics(capture_cold_start_metric=True)
def handler(event, context):
    body = json.loads(event['body'])

    # Process payment
    result = process_payment(body['amount'], body['payment_method'])

    # Publish event to EventBridge (fire and forget)
    eventbridge = boto3.client('events')
    eventbridge.put_events(Entries=[{
        'Source': 'payments.service',
        'DetailType': 'PaymentProcessed',
        'Detail': json.dumps({
            'orderId': body['order_id'],
            'userId': body['user_id'],
            'amount': body['amount'],
            'currency': body['currency'],
            'transactionId': result['transaction_id']
        }),
        'EventBusName': 'company-events'
    }])

    metrics.add_metric(name="PaymentProcessed", unit=MetricUnit.Count,
value=1)

    return {
        'statusCode': 200,
        'body': json.dumps({'transactionId': result['transaction_id']})
    }

```

EventBridge rule routing:

```

resource "aws_cloudwatch_event_rule" "payment_processed" {
    name          = "payment-processed"
    event_bus_name = "company-events"

    event_pattern = jsonencode({
        source      = ["payments.service"]
        detail-type = ["PaymentProcessed"]
    })
}

resource "aws_cloudwatch_event_target" "email_lambda" {
    rule          = aws_cloudwatch_event_rule.payment_processed.name
    event_bus_name = "company-events"
    arn           = aws_lambda_function.send_email.arn
}

# DLQ for failed events (emails that failed to send)
resource "aws_sqs_queue" "email_dlq" {
    name          = "email-lambda-dlq"
    message_retention_seconds = 1209600 # 14 days
}

```

Q64. How do you handle data migration and database modernization as part of cloud migration?

Answer:

Database modernization paths:

Legacy DB	Cloud Target	Migration Tool	Key Challenges
Oracle	Aurora PostgreSQL	AWS SCT + DMS	Stored procedures, Oracle-specific SQL
SQL Server	Aurora MySQL / RDS SQL Server	AWS DMS	Licensing cost → Aurora saves 90%
On-prem MySQL	Aurora MySQL	AWS DMS (homogeneous)	Minimal code changes
MongoDB	DocumentDB	mongodump + mongorestore	API compatibility check
Cassandra	Amazon Keyspaces	CQLSH copy	TTL behavior differences

Zero-downtime database migration pattern:

Phase 1: Set up DMS continuous replication
Source: On-prem Oracle → Target: Aurora PostgreSQL
DMS runs full load + ongoing CDC (Change Data Capture)

Phase 2: Cut over preparation

- Application code: update DB connection strings (use env var, not hardcoded)
- Test all queries on Aurora PostgreSQL in staging
- Measure performance: Aurora query vs Oracle query (indexes may differ)

Phase 3: Cut over (maintenance window = near-zero)

1. Put source DB in read-only mode (prevent new writes)
2. Wait for DMS to drain remaining changes (usually < 5 sec)
3. Update application DB endpoint → Aurora
4. Validate: run smoke tests
5. Enable writes on Aurora

Total downtime: < 30 seconds

Phase 4: Validation

- Row count comparison: source vs target (DMS validation task)
- Application error rate monitoring for 24 hours
- Keep DMS running reverse (Aurora → Oracle) for rollback capability for 7 days

Section 11: Leadership, Architecture Governance & Soft Skills {#section-11}

Q65. How do you write an Architecture Decision Record (ADR)?

Answer:

ADR template I use:

ADR-042: Service Mesh Selection for EKS Production Platform

****Date:**** 2025-03-15
****Status:**** Accepted
****Deciders:**** Platform Team, Security Team
****Technical Story:**** PLAT-1234

Context

We need a service mesh for our EKS clusters to provide:

- mTLS between services (security requirement for SOC2)
- Traffic management (canary deployments, circuit breakers)
- Observability (per-service latency, error rate metrics)
- 12 teams, 200+ microservices will onboard within 12 months

Decision Drivers

- Must support mTLS in STRICT mode (SOC2 requirement)
- Must expose Prometheus metrics per service (existing Grafana stack)
- Must support canary traffic splitting (Argo Rollouts integration)
- Must be maintainable by a 3-person platform team
- Upgrade path must be clear and non-disruptive

Options Considered

Option 1: Istio

****Pros:**** Feature-rich; strong community; Argo Rollouts native integration;

Kiali UI

****Cons:**** High operational complexity; CRD sprawl; large control plane footprint (2–4 CPUs, 4GB)

Option 2: Linkerd

****Pros:**** Extremely lightweight (< 100ms overhead); simple; secure by default

****Cons:**** Less feature-rich traffic management; no built-in circuit breaker; smaller community

Option 3: AWS App Mesh

****Pros:**** Managed control plane (no ops burden); native AWS integration

****Cons:**** Vendor lock-in; Envoy proxy only; less Kubernetes-native; AWS announced reduced investment

Decision

****Chosen: Istio****

Rationale: The team's requirement for advanced traffic management (canary via Argo Rollouts) and the existing investment in Prometheus/Grafana make Istio the best fit despite higher operational complexity. We mitigate complexity through:

- Using Helm for installation (not istioctl) for GitOps compatibility
- Ambient mode (sidecarless) on EKS 1.31+ – reduces overhead significantly
- Platform team owns Istio; application teams use pre-built VirtualService templates

Consequences

****Positive:****

- mTLS STRICT mode satisfies SOC2 CC6.7
- Per-service golden signals exported to existing Prometheus
- Canary deployments via Argo Rollouts + Istio VirtualService

****Negative:****

- 2–3 CPU, 2–4GB overhead for control plane (istiod)
- Sidecar injection adds ~50MB memory per pod
- Learning curve for application teams (mitigated with templates)

****Risks:****

- Istio upgrade complexity – mitigation: blue/green cluster upgrade strategy
- Sidecar version drift – mitigation: Istio upgrade controller (Kiali)

Review Date

Revisit in 12 months (Q1 2026) to evaluate Istio Ambient mode maturity.

Q66. How do you run a technical architecture review for a proposal from another team?

Answer:

Architecture review process:

1. Pre-review preparation (1 week before):

- Team submits RFC/ADR draft 5 business days in advance
- Reviewers read async; prepare questions/concerns in writing
- Define review scope: technology choice, security, scalability, operational concerns

2. Architecture review meeting (60-90 min):

Structure:

- 5 min: Proposing team presents problem statement (NOT solution yet)
- 10 min: Constraints review (budget, timeline, compliance, existing stack)
- 20 min: Proposed solution walkthrough
- 30 min: Questions and concerns from reviewers
- 10 min: Action items and next steps
- 5 min: Record outcome (Approved / Approved with conditions / Needs revision)

3. Evaluating proposals — my checklist:

Security:

- ☐ How is this authenticated and authorized?
- ☐ What data does it access? Is it encrypted?
- ☐ What happens when this service is compromised?
- ☐ Has secrets management been addressed?

Reliability:

- ☐ What are the failure modes? What happens if dependency X is down?
- ☐ Is there a circuit breaker? Retry logic with backoff?
- ☐ Are SLAs defined? Error budget allocated?
- ☐ Multi-AZ? Multi-region?

Operability:

- ☐ How will this be monitored? Metrics? Logs? Alerts?
- ☐ How do engineers on-call debug this at 2am?
- ☐ What is the runbook for common failure scenarios?
- ☐ Is there a rollback plan?

Cost:

- ☐ What is the estimated monthly cost at 10x current scale?
- ☐ Are there cost optimization opportunities?

Scalability:

- ☐ What is the expected growth rate?

- ☐ Where are the bottlenecks at 10x load?
- ☐ Are there stateful components that limit horizontal scaling?

4. Constructive pushback:

Not: "This design is wrong."

Yes: "I see a concern with the synchronous call chain – if the notification service is slow, it could cascade to the payment flow. Have you considered

making this async with EventBridge? What are your thoughts on that trade-off?"

Not: "Why didn't you use Kubernetes?"

Yes: "Help me understand the decision to use Elastic Beanstalk.

What factors led away from ECS/EKS? I want to make sure we're aligned on the long-term platform direction."

Q67. How do you mentor engineers and build a high-performing cloud/DevOps team?

Answer:

Mentoring framework I use:

1. Assess current competency (individual)

Skill assessment matrix (1–5 scale for each person):

Cloud (AWS): Compute | Networking | Storage | IAM | Data Services

IaC: Terraform | CloudFormation | Pulumi

Kubernetes: Deployment | Networking | Security | Operations

CI/CD: Pipeline design | Security integration | GitOps

Observability: Prometheus | Grafana | Alerting | Incident response

Use results to create individual development plans.

2. Deliberate skill development

Level 1 (learning): Pair programming with me on real tasks

Level 2 (practicing): Owns a real task with my code review

Level 3 (applying): Owns a full feature independently; I review design only

Level 4 (teaching): Mentors Level 2 engineers; leads architecture discussions

3. Learning resources I create:

- **Runbooks:** Detailed how-to for common tasks (setting up new EKS cluster, debugging CrashLoopBackOff)
- **Architecture office hours:** Weekly 30-min session; engineers bring real problems
- **Tech talks:** Monthly internal talks on new tools, post-mortems, architecture patterns
- **Blameless post-mortems:** Best learning tools — what failed, why, how to prevent

4. Measuring team growth:

DORA metrics (team output):

- Deployment frequency: from monthly to weekly to daily
- Lead time for changes: measure in sprint review
- Change failure rate: track post-deploy incidents
- MTTR: time to restore service

Qualitative signals:

- Engineers proactively raising architecture concerns (not waiting to be asked)
- Junior engineers reviewing PRs, not just submitting them
- Team owns their on-call without needing escalation for common issues

Q68. How do you handle a disagreement with a senior stakeholder about an architectural decision?

Answer:

Situation example: CTO wants to use a commercial SaaS monitoring tool (Datadog) at 400K/year. *I believe the open – source Grafana stack can deliver 9050K/year.*

My approach:

1. Understand their position before pushing back:

"Help me understand the decision drivers for Datadog.
Is it:

- Specific features not available in Grafana? (ML anomaly detection? APM?)
- Vendor support SLA requirements?
- Team familiarity and onboarding speed?
- Integration with existing tools?

I want to make sure we're evaluating the same criteria."

2. Present data, not opinions:

"Here's a comparison I put together:

Criteria	Grafana OSS	Datadog
Annual cost	\$50K (infra + support)	\$400K
Setup time	2 weeks	1 week
APM traces	Tempo + OTel	Native APM
ML anomaly	Manual setup	Built-in
Support	Community + commercial	24/7 enterprise
Vendor lock-in	Low (OTel-based)	High (proprietary agents)

The \$350K delta over 3 years = \$1.05M. Recommendation: Start with Grafana OSS

for 80% of use cases; budget Datadog trial for the specific APM/ML use cases where we need to validate the gap."

3. Propose a bounded experiment:

"Rather than committing to either fully, could we:

1. Deploy Grafana OSS this quarter (fast, low cost)
2. Run Datadog trial for 60 days on one critical service
3. Evaluate specific gaps with data, then decide

This limits the risk of a \$400K annual commit before we validate the value."

4. Accept the decision with conditions:

"If the decision is Datadog, I want to ensure we:

1. Use OTel for instrumentation (not Datadog agents) – preserves optionality
2. Negotiate a 1-year contract (not 3-year) given market alternatives
3. Define specific success metrics we expect in 12 months

I can fully support and implement either decision – I just want to ensure we're making it with eyes open."

Q69. You've just joined an organization with significant cloud technical debt. How do you prioritize and address it?

Answer:

Technical debt audit — first 30 days:

Week 1: Listen, observe, don't change anything

- Talk to every engineer about their pain points
- Read the last 10 post-mortems
- Review architecture diagrams (and where reality diverges)
- Look at cost reports, security findings, deployment frequency

Week 2: Quantify the debt

- Security: How many open Critical/High findings? (run Prowler/Wiz)
- Cost: Rightsizing opportunities? (Compute Optimizer)
- Reliability: MTTR? Deployment frequency? Change failure rate?
- Operational: Manual processes that should be automated?

Week 3: Classify and stack-rank

- Category A: Security vulnerabilities (immediate risk)
- Category B: Reliability issues (ongoing incidents)
- Category C: Operational inefficiency (developer experience)
- Category D: Cost waste (financial impact)

Week 4: Present roadmap to leadership

- Quick wins (1–4 weeks): Fix open security groups, enable MFA, tag resources
- Medium-term (1–3 months): Standardize CI/CD, enable CSP guardrails
- Long-term (3–12 months): Migrate to IaC, containerize legacy, implement GitOps

Prioritization matrix:

	High Risk	Low Risk
High Effort	Plan	Backlog
Low Effort	Do Now	Nice-to-have

Security vulnerabilities + low effort → DO NOW

Legacy arch + no security risk + high effort → PLAN with ROI

Never do: Re-architect everything at once. The goal is incremental improvement while keeping the lights on.

Q70. How do you manage the balance between speed (developer velocity) and safety (governance/compliance)?

Answer:

False dichotomy: Speed and safety are not opposites when done right. The real enemy is **toil** — repetitive manual work that slows teams AND introduces errors.

How I create "paved roads" (fast paths that are safe by default):

Golden path for new service:

1. Click "New Service" in Backstage (internal developer portal)
2. Select: language, team, compliance profile
3. Get in 5 minutes:
 - GitHub repo with golden pipeline pre-configured
 - Terraform workspace with VPC, ECR, IAM role
 - Helm chart template
 - Observability pre-configured (Prometheus, alerts, dashboards)
 - Security gates: Gitleaks, Trivy, OPA in CI

Developer effort: 0 security work needed on day 1

Platform team: writes the golden path once, all teams benefit

"You build it, you run it" with guardrails:

Teams own their services completely:

- Can deploy independently (no platform team bottleneck)
- Can modify their Terraform within approved module constraints
- Can configure their own alerts and dashboards

Platform team provides:

- The rails (modules, pipelines, policies)
- Not the traffic cop (not blocking every deploy)
- Support when teams hit the rails (not punishment for hitting them)

Where I draw hard lines:

Non-negotiables (always block):

- Secrets committed to git
- CRITICAL CVEs in production
- Public S3 buckets

- Missing MFA on production access

Soft guardrails (warn, team decides):

- HIGH CVEs (give 30 days to remediate)
- Missing tags (warn in CI, block after 60-day grace period)
- Test coverage below 70% (metric reported, not enforced)

Q71. Describe your approach to writing runbooks and documentation that engineers actually use.

Answer:

Problems with traditional documentation:

- ✗ Long Word documents no one reads (outdated in 3 months)
- ✗ Documented at architecture level but not operation level
- ✗ No searchability (where is the runbook for X?)
- ✗ Not tested – instructions that don't work in production

My documentation philosophy: Make the right thing easy to find and easy to do.

Runbook format I use:

Runbook: RDS Database High CPU

****Severity:**** P2 | ****SLO Impact:**** Potential latency increase | ****Owner:**** Platform Team

Symptoms

- CloudWatch metric RDSCPUUtilization > 90% for > 5 minutes
- Application response times > 1000ms (P99)
- Alert: `rds-high-cpu-payments-prod` in PagerDuty

Immediate Actions (< 5 minutes)

1. Open CloudWatch console → RDS → payments-prod-aurora
2. Check: Active connections, Freeable Memory, Read/Write IOPS
3. Run slow query log check:

```
```sql
SELECT query, exec_count, exec_time_avg_ms
FROM sys.statement_analysis
WHERE exec_time_avg_ms > 1000
ORDER BY exec_time_avg_ms DESC
LIMIT 20;
```

4. If single runaway query: **KILL <query\_id>;**
5. If connection storm: check connection pooling (PgBouncer/RDS Proxy)

## Escalation

If not resolved in 15 minutes: Page platform-team-secondary

## Post-Incident

- File JIRA ticket with slow query for optimization
- Check query plan: **EXPLAIN ANALYZE <slow query>**
- Consider: Read replica offload, index addition, query optimization

**\*\*Documentation standards I enforce:\*\***

1. **\*\*Runbooks live with the code\*\*** – In the repository, not Confluence
2. **\*\*Tested in staging\*\*** – If you can't run the runbook successfully in staging, it's not done
3. **\*\*Updated as part of incident resolution\*\*** – Post-mortem action item: "Update runbook X"
4. **\*\*Linked from alerts\*\*** – PagerDuty alert includes link to runbook
5. **\*\*Version controlled\*\*** – Git history shows who changed what and why



---

### Q72. How do you handle a production outage that reveals a systematic architectural flaw?

**\*\*Answer:\*\***

**\*\*Real scenario I handled:\*\***

Incident: Payment service went down for 23 minutes during Black Friday peak  
Root cause: All services connected directly to the same RDS instance. A long-running analytics query from the reporting service saturated the connection pool, starving the payment service of DB connections. MTTR: 23 minutes (kill analytics query, restore payments)

**\*\*Immediate (during incident):\*\***

1. Mitigate first: kill the analytics query, restore payments (23 min)
2. Don't fix root cause during the incident — that's how you make it worse
3. Document timeline in real-time (Slack thread + incident channel)

**\*\*Blameless post-mortem (48 hours later):\*\***

```markdown

Post-Mortem: Payments DB Connection Exhaustion (BF-2024-001)

Impact

- 23 minutes payments service unavailable
- ~\$340K lost revenue (estimated)
- 12,000 failed checkout attempts

Timeline

- 14:32 – Analytics team starts end-of-day sales report (large aggregate query)
- 14:41 – RDS connection count hits max_connections (200)
- 14:43 – Payment service begins returning 500 errors
- 14:45 – On-call paged; starts triage
- 14:52 – Root cause identified: analytics long-running query
- 14:55 – Analytics query killed; payments service recovers

Root Causes (5 Whys)

Why 1: Why did payments fail? → RDS connections exhausted

Why 2: Why were connections exhausted? → Analytics query held 180 connections

Why 3: Why did analytics have that many connections? → No connection limit

per service

Why 4: Why no connection limits? → Connection pooling never implemented

Why 5: Why no pooling? → "Never needed it" – single service initially, grew without revisiting

Systematic Flaw

All services share one RDS endpoint with no isolation between workloads. Long-running OLAP queries compete with OLTP payment queries.

Action Items

| Action | Owner | Due | Priority |
|---|----------------|---------|----------|
| Deploy RDS Proxy + per-service connection pools | Platform | 2 weeks | P0 |
| Separate analytics to read replica | Analytics team | 1 week | P0 |
| Add CloudWatch alarm: connections > 80% max | Platform | 2 days | P1 |
| Add circuit breaker in payment service | Payments team | 3 weeks | P1 |
| Review all services for connection pool isolation | Platform | 1 month | P2 |

The architectural fix (not just the incident fix):

Before:

All services → One RDS endpoint

After:

- └ RDS Proxy (OLTP) ← Payments, Identity, Orders
- └ Read Replica → Analytics, Reporting (read-only)

RDS Proxy:

- Per-service connection pool limits (payments: 50 max, analytics: 10 max)
- Multiplexes connections (100 app connections → 10 actual DB connections)
- IAM authentication (no more password-based connections)

Quick Reference Cheat Sheet — Multi-Cloud DevOps Architect

CICD Order: Secrets scan → SAST → Build → SCA → Image scan → Sign → Deploy

K8s Security: Private API → mTLS (Istio STRICT) → NetworkPolicy deny-all → OPA Gatekeeper

IaC Quality: terraform fmt → validate → Checkov (static) → conftest (plan) → Terratest

SLO Formula: $(\text{good_events} / \text{total_events}) * 100 \geq \text{SLO\%}$; Error Budget = $100\% - \text{SLO\%}$

Cost Priority: Spot (60% savings) → Savings Plans (30%) → Rightsize (20%) → Dev shutdown

Blast Radius: Account isolation → VPC segmentation → IAM least privilege → K8s NS → Feature flags

Zero Trust: No implicit trust → mTLS everywhere → Workload identity → Per-request AuthZ

DR Targets: RTO (how fast you recover) | RPO (how much data you can lose)

End of Part 2 | Combined: 82 Questions covering all JD requirements

Certification Roadmap for this Role

| Priority | Certification | Why It Matters |
|------------------------------|--|--|
| <div><div></div>High</div> | AWS Solutions Architect Professional | Primary cloud; validates advanced AWS design |
| <div><div></div>High</div> | CKA (Certified Kubernetes Administrator) | Core platform engineering skill |
| <div><div></div>Medium</div> | CKS (Certified Kubernetes Security Specialist) | Security + compliance requirements |
| <div><div></div>Medium</div> | Azure Solutions Architect Expert (AZ-305) | Secondary cloud requirement |
| <div><div></div>Medium</div> | HashiCorp Terraform Associate | IaC tooling validates depth |
| <div><div></div>Nice</div> | GCP Professional Cloud Architect | Third cloud; completes multi-cloud story |
| <div><div></div>Nice</div> | AWS DevOps Engineer Professional | Validates CI/CD + automation depth |